Animación y esqueletos

CC3501 Computación gráfica

2026



Nicolaes Eliaszoon Pickenoy: The osteology lesson of Dr. Sebastiaen Egbertsz (1619)

De la marioneta a la piel

Dado un esqueleto jerárquico en movimiento, ¿cómo deformamos una **malla continua** de forma convincente?

Marioneta rígida contra piel continua

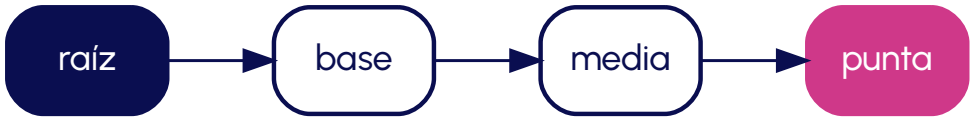
En grafos de escena armamos marionetas: piezas rígidas donde rotar el hombro arrastra el codo y la mano. Cada pieza es un objeto separado.

En un personaje moderno la geometría no está dividida: hay una **sola malla** que se dobla sin separarse, como la piel sobre los músculos. El plan de la clase es llegar de lo primero a lo segundo, paso a paso.

El esqueleto es un grafo de escena

Huesos, articulaciones y dos poses

Las **articulaciones** (*joints*) son los nodos y los **huesos** (*bones*) las aristas. Cada articulación guarda dos cosas:



- **pose de reposo** (*bind pose*): con el esqueleto quieto.
- **pose actual**: durante la animación.

La pose global de un hueso se compone con la de su padre (cinemática directa), igual que cualquier grafo de escena:

$$M_j = M_{\text{padre }(j)} \cdot T_j^{\text{local}}$$

La cadena de la raíz a la hoja

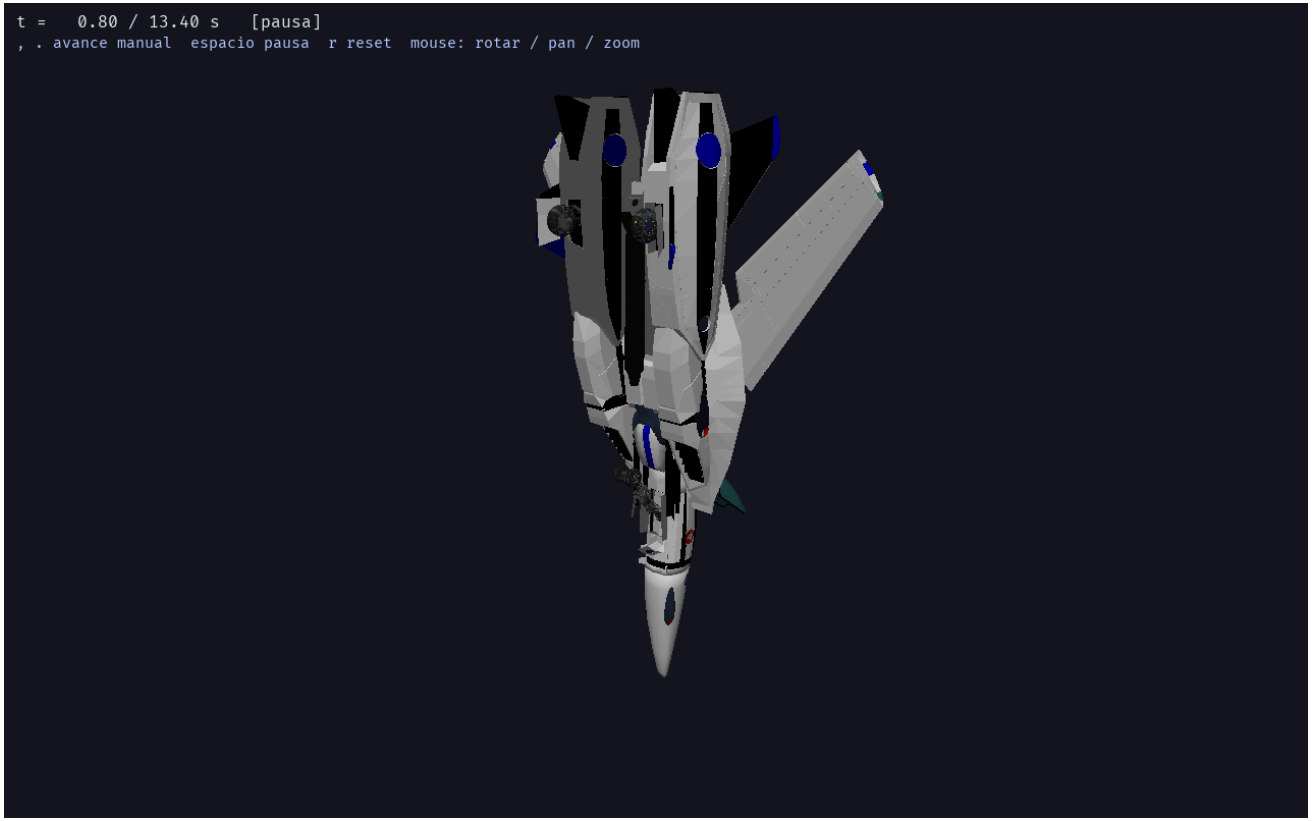
Cada hueso parte de la pose de su padre y le aplica su traslación y su rotación local, de la raíz a la hoja:

```
# pose global de cada hueso (forward kinematics)
m0 = tr.identity()                                          # raíz, fija
m1 = m0 @ tr.translate(0, salto, 0) @ tr.rotationZ(angulo_1)
m2 = m1 @ tr.translate(0, salto, 0) @ tr.rotationZ(angulo_2)
```



La raíz va a la izquierda porque su transformación envuelve a todo lo que cuelga de ella: `m0` arrastra a `m1` y `m1` arrastra a `m2`. Rotar `m1` mueve a `m2`, pero rotar `m2` no toca a `m1`. Por eso el orden va de la raíz hacia la hoja y no es `m2 @ m1 @ m0`.

Si `angulo_1` y `angulo_2` valen cero se recupera la pose de reposo: los huesos quedan alineados.



`animacion_esqueletica`. Un Valkyrie VF-1S de *Macross*. Cada panel y ala es una pieza rígida parenteada a un hueso; la animación mueve los huesos por *keyframes* TRS y cada pieza hereda su transformación global. Es solo composición de matrices: el caso de la marioneta, sin deformación.

Pegar la malla al esqueleto

Ida y vuelta por el hueso

Queremos que un vértice v siga a un hueso. El problema: v está en coordenadas de objeto, no del hueso. La solución es ida y vuelta:

1. La **inverse bind matrix** $M_{j,\,\text{rest}}^{-1}$ lleva v al espacio local del hueso en reposo.
2. La **pose actual** M_j lo devuelve al mundo, ya animado.

$$v' = M_j \cdot M_{j,\,\text{rest}}^{-1} \cdot v$$

La matriz combinada $M_j \cdot M_{j,\,\text{rest}}^{-1}$ es la **matriz de skinning** del hueso. En reposo $M_j = M_{j,\,\text{rest}}$, así que vale la identidad y nada se deforma.

Una matriz de skinning por hueso

La inverse bind se calcula una vez al inicio (es la inversa de la pose de reposo). La matriz de skinning se arma cada cuadro:

```
# una sola vez: del mundo en reposo al espacio local de cada hueso
INVERSE_BIND = [np.linalg.inv(pose) for pose in poses_de_reposo]

# cada cuadro: matriz de skinning por hueso = pose actual @ inverse bind
skin = [globales[j] @ INVERSE_BIND[j] for j in range(n_huesos)]
```

Un vértice, varios huesos: LBS

Mezcla ponderada

Cerca de una articulación un vértice no pertenece a un solo hueso. Le damos **pesos** w_j que suman 1 y promediamos las matrices de skinning:

$$v' = \sum_j w_j \cdot M_j \cdot M_{j,\,\text{rest}}^{-1} \cdot v$$

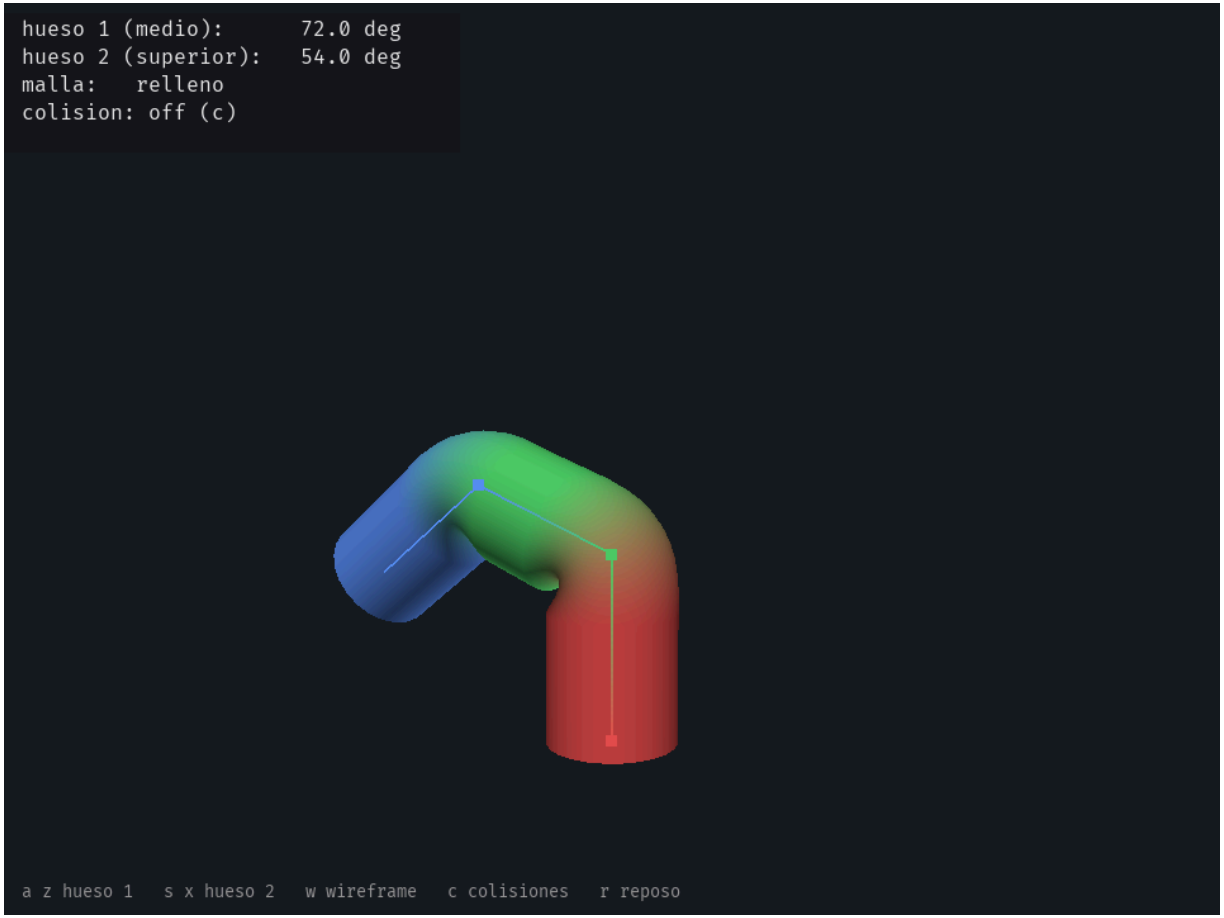
Es **Linear Blend Skinning**: combinación lineal de las posiciones que daría cada hueso. Los pesos se pintan en herramientas como Blender y viajan como atributos del vértice.

La mezcla en el vertex shader

La GPU mezcla las matrices con los pesos del vértice y transforma la posición. Hasta cuatro huesos por vértice:

```
mat4 skin_matrix = mat4(0.0);
for (int i = 0; i < 4; i++)
    skin_matrix += weights[i] * skin_matrices[int(joints[i])];

vec4 skinned_pos = skin_matrix * vec4(position, 1.0);
```



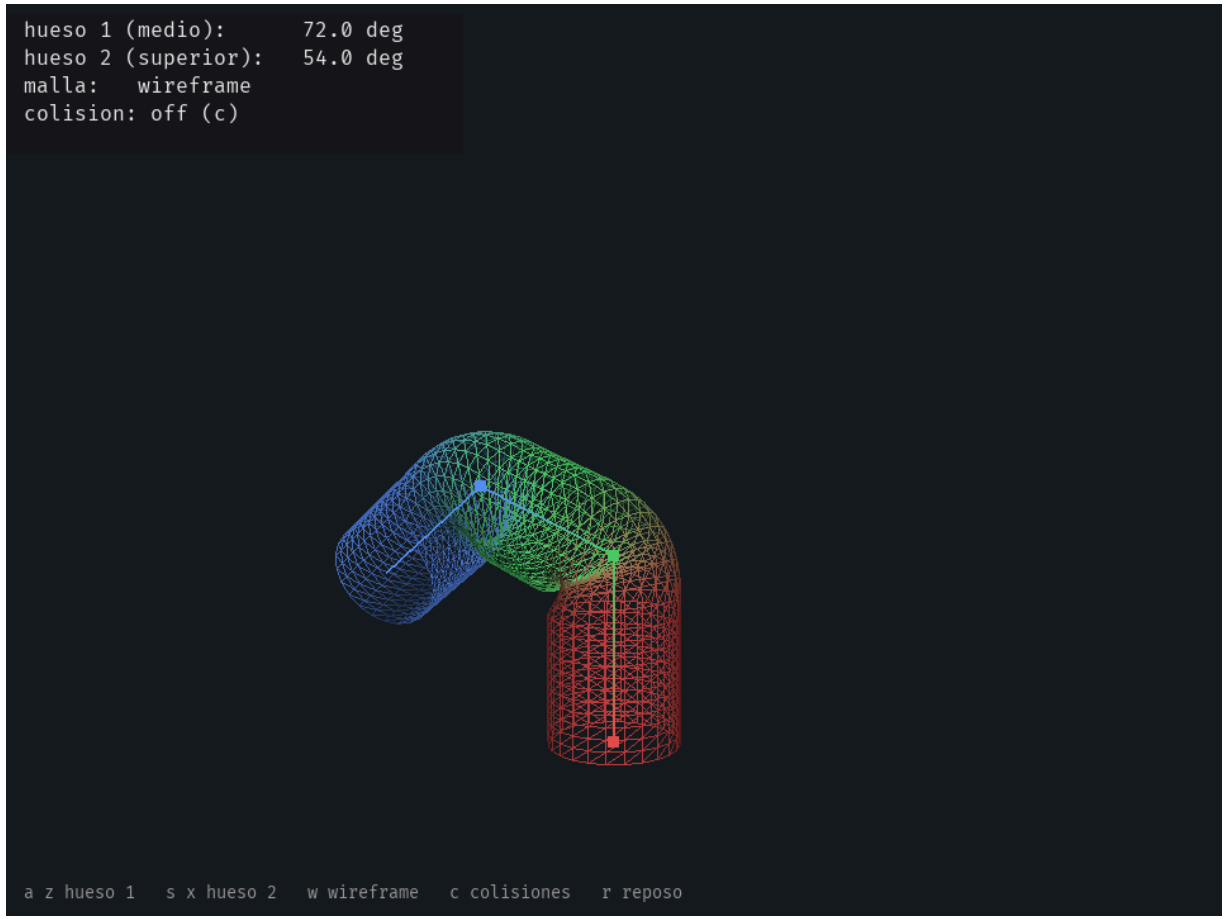
`esqueleto_lbs`. Un tubo gobernado por tres huesos, sin cargar archivos. La base está fija y dos articulaciones se rotan con el teclado. El tubo se tinta con la mezcla de un color por hueso ponderada por los pesos: las bandas de transición (rojo a verde a azul) se ven doblarse junto al esqueleto.

Las normales también se deforman

La normal es una dirección: la traslación no la afecta y, si la transformación no es rotación pura, aplicarla tal cual la deja torcida. Se usa la **matriz de normales** (inversa traspuesta de la parte 3×3):

```
mat3 normal_matrix = transpose(inverse(mat3(skin_matrix)));
vec3 skinned_normal = normalize(normal_matrix * normal);
```

La mezcla M_{skin} no es una rotación aunque cada M_j lo sea, así que la inversa traspuesta importa justo en las articulaciones. Sin ella el sombreado se ensucia en codos y rodillas.



`esqueleto_lbs`, *wireframe* (tecla `w`). La triangulación deja ver la deformación: los triángulos se estiran en la cara exterior de cada curva y se comprimen en la interior.

Dónde corre cada cosa

CPU y GPU se reparten el trabajo

La CPU arma una matriz por hueso cada cuadro y la sube como un *uniform array*. La malla (posiciones, pesos, índices de hueso) ya vive en la GPU; solo viajan las matrices.

En la CPU se empaqueta el arreglo y se sube:

```
# una matriz 4×4 por hueso, aplanada a 16 floats por columnas (orden de GLSL)
skin_view[:] = np.array(skin).transpose(0, 2, 1).reshape(n_huesos, 16)
pipeline["skin_matrices"] = skin_buffer
```

En el shader es un arreglo de matrices, junto a los índices y pesos de cada vértice:

```
uniform mat4 skin_matrices[91];   // una por hueso (91 en el zorzal)

in vec4 joints;    // 4 indices de hueso por vertice
in vec4 weights;   // sus 4 pesos (suman 1)
```

El *vertex shader* usa cada índice de `joints` para leer su matriz del arreglo `skin_matrices` y las combina con `weights`, la mezcla que vimos antes.

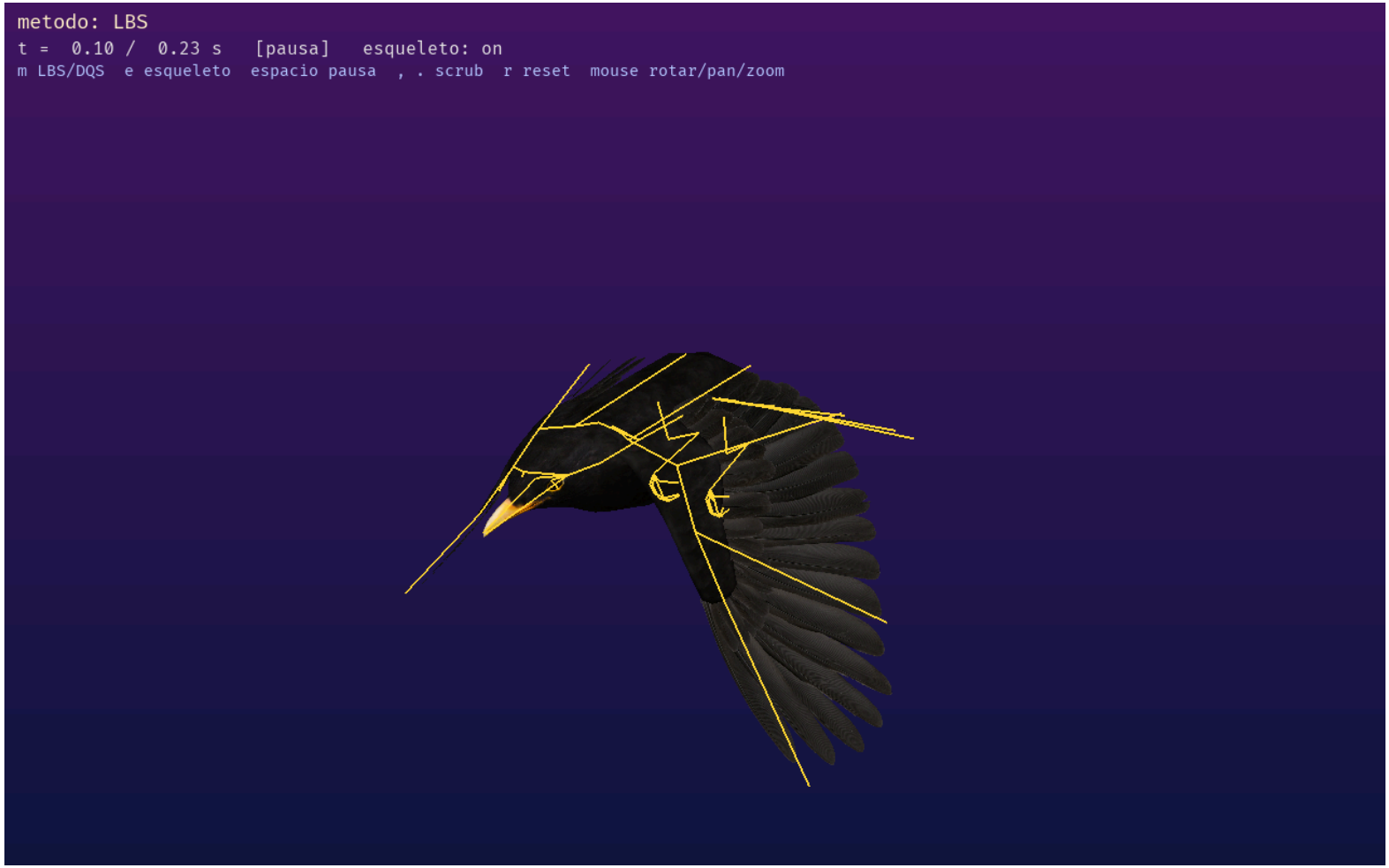
Cuando LBS falla

Artefactos del LBS

La suma ponderada de rotaciones no es una rotación. De ahí sus defectos:

- ***Candy-wrapper***: al torcer 180° el volumen colapsa.
- **Pérdida de volumen**: codos y rodillas se adelgazan al doblarse.

Dual Quaternion Skinning (DQS) representa cada hueso como un par de cuaterniones, los mezcla en la esfera de cuaterniones y renormaliza. El resultado siempre es rígido: conserva mejor el volumen, a más costo por vértice.



skinning. Un zorzal con animación de aleteo y el esqueleto visible (tecla `e`). La misma malla, los mismos huesos y los mismos pesos se deforman con LBS o DQS (tecla `m`). Cada vértice mezcla hasta cuatro huesos.

El movimiento en el tiempo

Keyframes e interpolación

Los animadores definen poses en *keyframes* y el sistema interpola entre ellas. Interpolar matrices de rotación no funciona; se usan **cuaterniones unitarios** y SLERP:

$$\text{slerp}\ (q_0, q_1, t) = q_0 \cdot \left(q_0^{-1} \cdot q_1\right)^t$$

Da una rotación a velocidad angular constante por el arco más corto. Los ***blend trees*** combinan varias animaciones según parámetros continuos, por ejemplo mezclar "caminar" y "correr" según la velocidad.

Promediar matrices no da una rotación

Interpolar las matrices entrada por entrada parece lo natural, pero el promedio de dos rotaciones deja de ser ortonormal: encoge y deforma en vez de rotar.

```
# INCORRECTO: promedio lineal entrada por entrada
def lerp_matrices(R0, R1, t):
    return (1 - t) * R0 + t * R1

# a mitad de camino entre 0 y 180 grados en torno a Z:
R0 = np.eye(3)
R1 = np.diag([-1, -1, 1])
R  = lerp_matrices(R0, R1, 0.5)   # → diag(0, 0, 1), con det = 0
```

A la mitad del giro, R aplasta el plano xy a una línea y el objeto desaparece. Las rotaciones forman una superficie curva (la esfera de cuaterniones unitarios): el promedio lineal corta por dentro y se sale de ella. SLERP avanza sobre la superficie, así que siempre entrega una rotación válida, aquí la de 90°.

SLERP por el arco más corto

Un cuaternión q y su negado $-q$ representan la misma rotación. Por eso el código elige primero el signo que da el arco corto (producto punto positivo) y recién entonces aplica los pesos seno de la fórmula:

```
def slerp(q0, q1, t):
    dot = np.dot(q0, q1)
    if dot < 0.0:                  # arco corto: q y -q son la misma rotacion
        q1, dot = -q1, -dot
    if dot > 0.9995:               # casi paralelos: lerp y renormaliza
        q = q0 + t * (q1 - q0)
        return q / np.linalg.norm(q)
    theta_0 = np.arccos(dot)       # angulo entre q0 y q1
    s0 = np.sin((1 - t) * theta_0) / np.sin(theta_0)
    s1 = np.sin(t * theta_0)        / np.sin(theta_0)
    return s0 * q0 + s1 * q1
```

`grafica.gltf` usa esta forma para interpolar las rotaciones TRS entre *keyframes* al cargar una animación glTF.

Colisiones con una malla que cambia

La estructura de colisión queda vieja

El *skinning* cambia la forma de la malla cada cuadro. Toda estructura calculada en la pose de reposo se invalida apenas se mueve el esqueleto: la AABB del modelo deja de contenerlo, una OBB por PCA deja de calzar, un BVH queda con límites viejos.

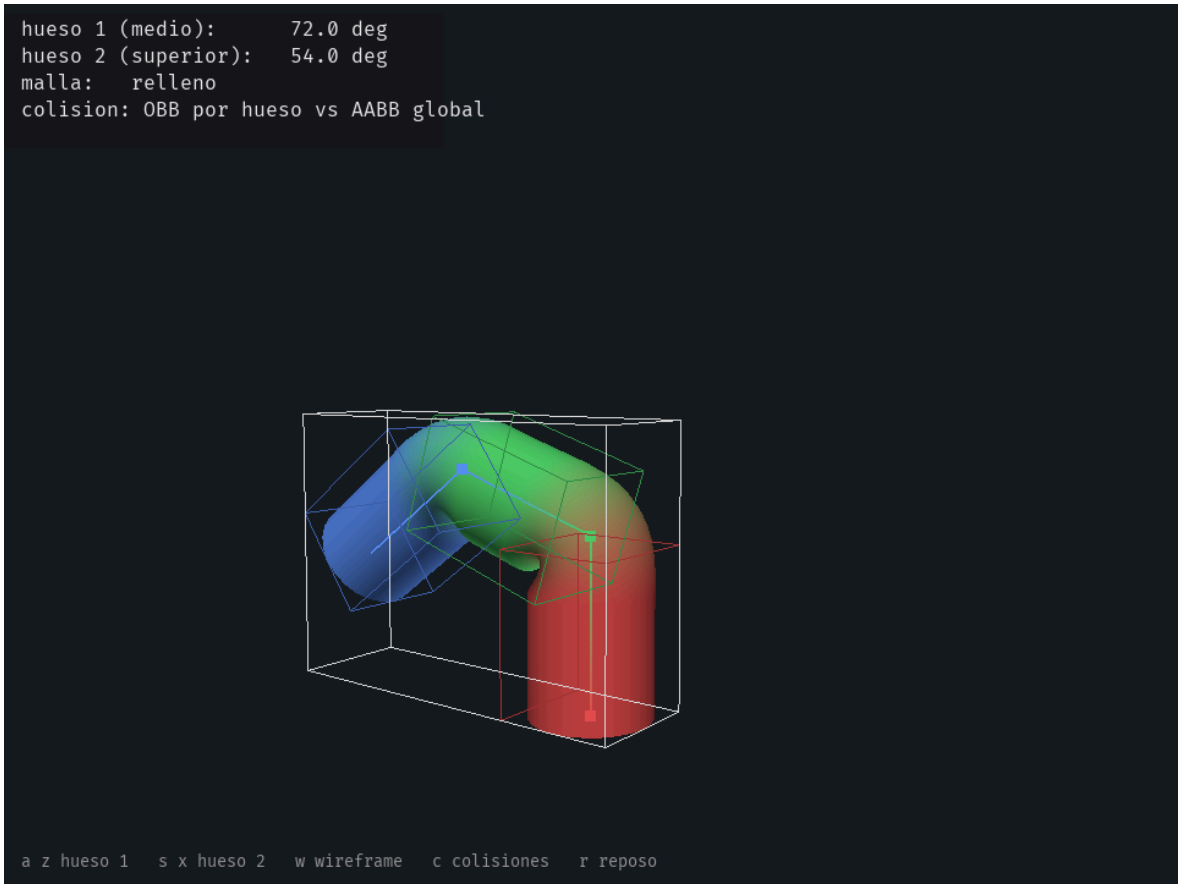
Recalcular todo cada cuadro (re-*skinning* en CPU más reconstruir el árbol) paga el costo del *skinning* dos veces.

Un volumen rígido por hueso

En vez de encerrar la malla, se pega un volumen rígido a cada hueso, definido en su espacio local. No se recalcula: solo lo mueve la pose del hueso, igual que la OBB de la unidad de colisiones.

```
# la caja vive en el espacio local del hueso; la pose actual la mueve
esquinas_mundo = (globales[j] @ esquinas_locales.T).T
```

La **cápsula** es la primitiva habitual (un hueso es alargado): son las *hitboxes* y los *ragdolls* de los videojuegos.



`esqueleto_lbs`, colisión (tecla `c`). Cada hueso lleva una OBB en sus colores, ajustada porque sigue su pose. La caja gris es la AABB de toda la malla, recalculada cada cuadro: al doblar el tubo se infla y deja mucho espacio vacío.

Síntesis

El esqueleto es la estructura barata que gobierna todo: deforma la malla (*skinning*), interpola el movimiento (*keyframes*) y conduce la colisión (un volumen por hueso).

¡Gracias!

egraells@dcc.uchile.cl